ModIntel: Intelligent Web Application Firewall

Comprehensive Test Plan Document (TPD) & Final Evaluation Report

Project Title: ModIntel – AI-Powered Web Application Firewall with Machine Learning Anomaly Detection
Document Version: 1.4
Date: May 26, 2026
Standards Compliance: IEEE Standard for Software Test Documentation (IEEE Std 829-2008)
Submitted for: Final Year Project Evaluation

---

1. Introduction

1.1. Executive Summary

ModIntel is an advanced, intelligent web application firewall (WAF) and real-time monitoring system designed to secure modern web applications against complex cyber threats. By integrating traditional signature-based inspection (powered by ModSecurity/Coraza) with microservice-driven machine learning models, ModIntel accurately identifies both known vectors and zero-day vulnerabilities without compromising request latency.
The platform is structured as a resilient distributed system consisting of multiple core microservices:

- **Authentication Service (Go):** Manages user sessions, access controls, and security tokens.
- **Review API (Go):** Handles security event logs, alert pagination, and metadata distribution.
- **Log Collector & Health Aggregator (Go):** Consolidates telemetry, pings operational nodes, and routes audit trails.
- **ML Inference Engine (Python):** Orchestrates high-speed payload feature extraction, anomaly scoring, and model execution.

1.2. Purpose of the Document

The purpose of this Test Plan Document (TPD) is to establish a rigorous, formal framework detailing the testing scope, strategy, resource allocation, and execution results for the ModIntel platform. It ensures that all distributed services adhere strictly to functional requirements, architectural designs, and non-functional performance benchmarks before deployment. This iteration acts as an integrated report combining foundational test targets with their verified empirical outcomes as of May 2026.

2. Features to be Tested vs. Not Tested

2.1. Features to be Tested

- **User Authentication & Security:** Validation of JWT token generation, cryptographic password hashing, API rate

limiting, and Role-Based Access Control (RBAC) enforcement.

- **Microservice REST APIs:** Integrity of Review API endpoints, cursor-based alert pagination, and ruleset metadata configuration retrieval.
- **Machine Learning Inference Pipeline:** String feature extraction (MissModelFeatureExtractor), ONNX model prediction loops, real-time anomaly scoring, and mathematical model calibration metrics.
- **Inter-Service Integration:** Isolated cross-container network communication, database read/write cycles, and centralized aggregate health metrics tracking.
- **WAF Gateway Traffic Flow:** Reverse-proxy routing via the Caddy server, Coraza Core Rule Set (CRS) / ModSecurity rule execution, and physical mitigation/blocking of structural vectors including SQL Injection (SQLi), Cross-Site Scripting (XSS), and Path Traversal.

2.2. Features Not to be Tested

- **Third-Party Managed Service Availability:** Upstream cloud-provider hosting infrastructure uptime guarantees (e.g., managed MongoDB Atlas database availability clusters).
- **Frontend Dashboard UI Rendering:** Extended multi-browser CSS/HTML visual regression layout tests, limiting client-side testing purely to functional route reachability and live data rendering.
- **Physical Infrastructure Layers:** Low-level host Operating System kernel settings and hardware layer bare-metal load balancers.

3. Testing Strategy & Pyramid Approach

ModIntel employs a strict **Testing Pyramid** approach to establish deep test coverage across isolated, integrated, and full-stack operational layers.

- **Unit Testing:** Isolated tests on individual Go and Python functions (e.g., JWT parsing, feature extraction) using mock data. Executed via go test and pytest.
- **Integration Testing:** Tests ensuring that internal microservices interact correctly (e.g., Auth service reading from MongoDB, Log collector sending data to the Inference Engine). Executed via go test -tags=integration within a Docker Compose environment.
- **System Testing:** End-to-End (E2E) tests simulating real user and attacker traffic entering through the public WAF gateway, verifying the entire pipeline from attack detection to dashboard alert generation. Executed via go test -tags=system.

4. Pass/Fail Operational Criteria

4.1. Pass Criteria

A test profile is definitively categorized as a **Pass** when it satisfies all the conditions below:

1. The programmatic actual output mirrors the predefined expected outcome exactly down to individual data fields.

2. The targeted system demonstrates memory/thread safety and zero unhandled core crashes under peak simulated loads.
3. The server boundary returns precise HTTP status codes relative to the interaction context (e.g., 200 OK for authorized retrieval, 401 Unauthorized for bad credentials, and 403 Forbidden for blocked cyber threats).

4.2. Fail Criteria

A test execution is classified as a **Fail** if it hits any of these system thresholds:

1. An unhandled exception occurs, or an internal fault breaks execution flow, leading to unexpected 500 Internal Server Error responses.
2. Access controls fail, resulting in unauthorized data visibility or vertical privilege escalation.
3. The WAF edge layer or ML model fails to stop a known malicious payload, leaking unsafe input to internal backends.
4. Output payloads deviate structurally or textually from test definitions.

5. Formal Test Specifications & Manual Matrix

Table 1: Test Case Specification for User Authentication (Unit Layer)

Purpose: Verify that the authentication service validates credentials correctly and blocks unauthorized access.

Input Profile	Expected Behavioral Result	Test Parameters / Data Used	Actual Output Delivered	Status
All valid fields	Login successful, token generated	Email: admin@modintel.local Password: ChangeMe123	"Login successful, token generated"	**Pass**
Invalid Password, other fields valid	Invalid credentials	Email: admin@modintel.local Password: WrongPass99!	"Invalid credentials"	**Pass**
Email not registered / invalid format	Invalid credentials	Email: bad_actor@gmail.com Password: ChangeMe123	"Invalid credentials"	**Pass**
Empty Email or Password	Bad Request: Missing fields	Email: [Empty] Password: ChangeMe123	"Bad Request: Missing fields"	**Pass**

Table 2: Test Case Specification for AI Inference Prediction (Unit Layer)

Purpose: Ensure the ML model correctly extracts features and assigns an attack probability to HTTP requests.

Input Profile	Expected Behavioral Result	Test Parameters / Data Used	Actual Output Delivered	Status
Normal Traffic Features	Attack Probability < 0.2 Priority: None	URI: /index.html Body: empty	Attack Probability: 0.05	**Pass**
SQLi Payload Features	Attack Probability > 0.8 Priority: P1 or P2	URI: /search?q=1' OR 1=1--	Attack Probability: 0.92 Priority: P1	**Pass**
Empty URI / Missing Features	Validation Error	Malformed data fields	"Validation Error"	**Pass**

Table 3: Test Case Specification for Inter-Service Communication (Integration Layer)

Purpose: Verify that the Health Aggregator can successfully ping all dependent microservices.

Input Profile	Expected Behavioral Result	Test Parameters / Data Used	Actual Output Delivered	Status
All containers running	All services return status OK	HTTP GET to /api/health/aggregate	"review-api: ok proxy-waf: ok, auth: ok"	**Pass**
Database offline	Database shows as degraded	Kill MongoDB container	"review-api: degraded"	**Pass**

Table 4: Test Case Specification for API Data Retrieval (Integration Layer)

Purpose: Verify the Review API retrieves alerts from the database using cursor-based pagination.

Input Profile	Expected Behavioral Result	Test Parameters / Data Used	Actual Output Delivered	Status
Valid Limit, Empty Cursor	Returns first N records + next cursor	Limit: 50, Cursor: [Empty]	"50 records returned"	**Pass**
Valid Limit, Valid Cursor	Returns next N records after cursor	Limit: 25, Cursor: 64f1a2...	"25 records returned"	**Pass**
Invalid Limit (Too High)	Bad Request: Limit exceeded	Limit: 1000	"Bad Request: Limit exceeded"	**Pass**

Table 5: Test Case Specification for End-to-End Gateway WAF Routing (System Layer)

Purpose: Verify that the WAF Proxy blocks malicious traffic while routing normal user dashboard requests correctly.

Input Profile	Expected Behavioral Result	Test Parameters / Data Used	Actual Output Delivered	Status
Normal Traffic via WAF	"200 OK or 3xx Redirect"	GET /dashboard/overview.html	"200 OK"	**Pass**
SQLi Payload via WAF	"403 Forbidden (Blocked by Coraza/ModSec)"	GET /search?q=1%27%20OR%201%3D1	403 "Forbidden"	**Pass**
XSS Payload via WAF	"403 Forbidden"	GET /page?q=<script>alert(1)</script>	"403 Forbidden"	**Pass**
Full Auth Flow via Gateway	"Flow completed successfully"	Valid credentials /logout	"Flow completed successfully"	**Pass**

6. Detailed Unit Test Results – Python ML Components

This section presents comprehensive automated execution results for the core Machine Learning pipelines.

Table 6: Automated Module Verification Matrix

Module Name	Test File Reference	Test Cases	Passed	Coverage Focus
Feature Extraction	test_miss_feature_extractor.py	8	**8**	High (String token processing, payloads, math anomalies)
Data Preparation	test_prepare_balanced_dataset.py	3	**3**	Medium (Under-sampling ratios, matrix generation)
Training Helpers	test_train_miss_from_review.py	4	**4**	High (Real-time log scraping, weight output)
Schema Integrity	test_generate_schema_hash.py	3	**3**	Medium (Dynamic API changes, contract hashing)
TOTALS	**4 Core Pipeline**	**18**	**18**	**Excellent / Passed**

Module Name	Test File Reference	Test Cases	Passed	Coverage Focus
	Modules			

6.2. Detailed Breakdown – MissModelFeatureExtractor

- test_init: Constructor initialization validated. Pathing and default states verified.
- test_fit_and_feature_names: Signature vocabulary loaded correctly; mapped exactly **43 distinct structural feature fields**.
- test_transform_single_dict: Verified single log conversions preserve tensor array shapes and exact float types.
- test_transform_list: Batch transformation loops executed successfully without memory leaks.
- test_shannon_entropy: Validated mathematics behavior during string randomness testing, protecting edge-cases.
- test_evasion_detection: Confirmed catch mechanics for payload encoding bypass vectors (double encoding, hex mutations).
- test_sql_detection: Keyword counting metrics hit 100% verification accuracy when identifying raw injection text patterns.
- test_save_load: Model persistence validated using standard Joblib file serialization loops.

7. Unit Test Results – Go WAF Blocker

All **7 native Go automated tests passed** successfully.
Key Achievement: The core gateway interception pipeline (pre-filtering layers combined with external machine learning classification webhooks) achieved deep code-path coverage with zero flaky execution threads.

8. Integration and System Test Results Addendum

Table 7: Functional Integration & System Results Log

ID	Component	Test Case Description	Expected Result	Actual Empirical Result	Status
INT-01	Log Collector	Webhook log ingestion from Coraza/ModSec	Logs successfully parsed and stored in MongoDB	Logs parsed and stored within 50ms	**Pass**

ID	Component	Test Case Description	Expected Result	Actual Empirical Result	Status
INT-02	Review API	WAF rules metadata synchronization	API returns latest Coraza rules metadata	Correct JSON returned with 200 OK	**Pass**
INT-03	Inference Engine	AI Advisory prediction on malicious payload	Returns confidence score > 90% for SQLi	Returned 95.2% confidence score	**Pass**
INT-04	Auth Service	User login and JWT generation	Valid JWT returned upon correct credentials	Valid JWT generated and verified	**Pass**
SYS-01	Full Stack	End-to-end alert dashboard update	New attack log appears on dashboard < 1s	Alert visible on UI in ~300ms	**Pass**
SYS-02	Inference Engine	AI inference fail-open behavior	WAF continues logging if AI service is down	Logs processed without AI advisory	**Pass**

9. Performance Test Results (Non-Functional Requirements)

These tests validate the system's ability to handle load and deliver low-latency responses under enterprise-scale loads.

Table 8: System Performance Matrix

Test ID	Performance Metric	Target / Requirement	Actual Empirical Result	Status
PERF-01	API Latency (Review API - GET /api/alerts)	< 100ms (95th percentile)	45ms (p95)	**Pass**
PERF-02	AI Inference Latency	< 50ms per request	32ms per request	**Pass**
PERF-03	Log Ingestion Throughput	> 1000 logs/sec	1,450 logs/sec max	**Pass**
PERF-04	Dashboard UI Load	< 1s for initial	0.8s	**Pass**

Test ID	Performance Metric	Target / Requirement	Actual Empirical Result	Status
	Time	render		
PERF-05	Concurrent Connections (WebSockets)	Handle 100+ concurrent clients	Handled 250 stable connections	**Pass**

10. Challenges Faced and Lessons Learned

- ---

  Windows Compatibility Issues with Go Test Binaries: Intermittent compilation and lock issues solved cleanly using the -count=1 flag during runtime commands.
- **Floating Point Precision in Metrics:** Unit test assertions encountering fractional variances were converted to leverage delta approximation bounds via pytest.approx.
- **Hardcoded Paths in Data Scripts:** Rewritten entirely to utilize dynamic environment configuration strings, guaranteeing zero environment-specific pipeline execution drops.
- **Test Flakiness Due to Database File Locks:** Remedied by implementing deterministic setup and teardown commands (dropping collection sandboxes between test blocks).

11. Conclusion and Recommendations

The evaluation cycle of the ModIntel platform has been completed successfully. Core systems — specifically the traditional signature pre-filter layer combined with the machine learning prediction scoring system — are thoroughly backed by production-ready automated verification tools.

Strategic Next-Phase Roadmap Recommendations:

1. Fully transition the orchestration platform to run directly within centralized GitHub Actions CI/CD pipeline automation workflows.
2. Implement property-based testing matrices to thoroughly evaluate string permutations hitting the ML feature extractor.
3. Scale system load stress tests beyond current limits to evaluate cluster stability at 5,000+ concurrent connections.
4. Integrate automated security scanning dependencies (such as OWASP ZAP and Trivy container image audits) directly into standard branch protection rules.

Overall Project Testing Verdict: PASSED with Strong Confidence

12. References

[1] IEEE Standard for Software Test Documentation, IEEE Std 829-2008.

[2] ModIntel Architecture and API Documentation (Internal Source Materials).

[3] Scikit-Learn Engine and Go Testing Native Documentation Packages.